

AgentExchange_Solution_Intake_Questionnaire

AgentExchange Solution Intake Questionnaire — DocGen v2.1.0

Source form: https://resources.docs.salesforce.com/re11/doc/en-us/static/pdf/AgentExchange_Solution_Intake_Questionnaire.pdf **Submitted for:** Portwood DocGen Managed v2.1.0 (04tVx000000Zw5xIAC) **Submission date:** 2026-05-22

Applicability statement

The instructions on the questionnaire read:

“If the solution you’re submitting for AppExchange security review contains Agentforce elements, complete this questionnaire and submit it with your review.”

Portwood DocGen v2.1.0 ships no Agentforce metadata. The package contains:

- 0 GenAiPlugin (agent topic) metadata files
- 0 GenAiPromptTemplate (prompt template) metadata files
- 0 GenAiFunction (agent action) metadata files
- 0 Agentforce agents or agent extensions

However, DocGen ships 4 @InvocableMethod Flow actions (DocGenFlowAction, DocGenBulkFlowAction, DocGenGiantQueryFlowAction, DocGenSignatureFlowAction). Salesforce permits a subscriber admin to expose any Flow invocable as an Agentforce action by manually creating a GenAiFunction in their own org that wraps the Flow. We treat those four invocables as **potential** Agentforce action surfaces for the purposes of this questionnaire, even though the package itself does not wrap them. Question 5 below documents each one with its inputs, outputs, sharing model, and security gate so the reviewer can evaluate the “Maintain Trust with Agentforce Actions” criteria against the actual surface.

Everything else on the questionnaire (MIAW, third-party LLMs, agent topics, unmanaged Agentforce metadata) is not applicable. DocGen is a 100% native Salesforce document generation engine (PDF, Word, Excel, PowerPoint) with built-in electronic signatures, no callouts, no Models API usage, no MIAW integration.

Question 1 — Extension package status

“If the submitted package is an extension package: Has the base package passed security review? If the base package is also in the security review queue, share its Solution name / Package ID / Package version ID.”

Answer: N/A — not an extension package.

DocGen v2.1.0 is a **standalone** Managed 2GP package. It does not extend any other managed package. There is no base package dependency.

Field	Value
Solution name	Portwood DocGen Managed
Package ID	0Hoal00000003d9hCAA
Package version ID	04tVx0000000Zw5xIAC
Package type	Managed 2GP (standalone, not an extension)
Namespace	portwoodglobal
Base package	None

Question 2 — Dependency installation

“Are all dependencies installed and configured in the org submitted for security review?”

Answer: N/A — no Agentforce dependencies.

DocGen has no Agentforce dependencies, no external package dependencies, no Managed Packages installed alongside it, and no Subscriber-installed prerequisites beyond standard Salesforce platform features:

- The Spring '26 **Visualforce PDF Rendering Service** Release Update must be enabled in the subscriber org (documented in our post-install guide). This is a platform feature, not a package dependency.

The AppExchange Security Review Dev Org (dave.2a1209f2e79c@agentforce.com) has v2.1.0 installed and the post-install configuration completed (DocGen_Admin permset assigned, sample data seeded).

Question 3 — Agent type and topics

“What agent type does the solution depend on? If the solution requires multiple agents, list the topics associated with each agent.”

Answer: N/A — no agent dependency.

DocGen depends on no Agentforce agent. The package contains:

- 0 GenAiPlugin metadata files
- 0 GenAiPromptTemplate metadata files
- 0 GenAiFunction metadata files
- 0 agent topics
- 0 agent actions

Verified by source-tree search: `grep -ril "agentforce|GenAiPlugin|GenAiPromptTemplate|GenAiFunction|Models API" force-app/main/default/` returns no matches.

Question 4 — End-to-end use cases configured in the review org

“Is the agent configured and set up with end-to-end use cases in the org submitted for review?”

Answer: N/A — no agent in the solution.

For non-Agentforce end-to-end verification: yes, the AppExchange Security Review Dev Org has v2.1.0 installed, sample data seeded (Acme Demo Corp + 3 Contacts + 1 Opportunity + 3 Sample DocGen templates with attached DOCX bodies), and the running user assigned the DocGen_Admin permission set. The reviewer can exercise:

- Single-record document generation via the **DocGen Runner** LWC on the Acme account
- Bulk generation via the **DocGen Bulk Runner** (filter on Industry=Technology returns the 1 Acme record)
- Multi-signer signature request via **DocGen Signature Sender** quick action on the Acme record
- Public document verification via the **DocGen Authenticator** LWC and the /apex/DocGenVerify Visualforce page

See Listing_Describe_Solution.md (this folder) for the full architecture summary and SECURITY_REVIEW_RESPONSE_v2.md (at repo root) for the per-finding map of the v1.56 review.

Question 5 — Public agent actions

“Provide details about all public agent actions in the solution. See Maintain Trust with Agentforce Actions.”

DocGen ships **zero GenAiFunction records** (no packaged agent actions). It does ship **four @InvocableMethod Flow actions** that an admin in the subscriber org could wrap as agent actions by manually creating a GenAiFunction referencing one of them. Each invocable is listed below with the trust posture an admin would need to evaluate before exposing it as an agent action.

Action Name	Action Type	Sharing	Why the A
DocGenFlowAction.generate	Apex @InvocableMethod (Flow action)	global with sharing	Generates a + record. In Document Document is the canon document permset (ob
DocGenBulkFlowAction.generateBulk	Apex @InvocableMethod (Flow action)	global with sharing	Starts a Doc Inputs: Ten Combined ID, Error Flows use i (creates a D Schema.s0
DocGenGiantQueryFlowAction.generateDocument	Apex @InvocableMethod (Flow action)	global with sharing	Auto-detect heap limit; when not. I Outputs: C Query, Err Flows need knowing th
DocGenSignatureFlowAction.generate	Apex @InvocableMethod (Flow action)	global with sharing	Creates a D and returns Record Id Id), Send E Signing 0 URLs [], Si Success, E “Opportuni DocGen_Ad DocGenSig which runs Schema.s0 the LWC se

Trust posture (applies to all four invocables — for the “Maintain Trust with Agentforce Actions” criteria)

Criterion	DocGen invocables
Authentication	The Flow runs as the authenticated invoking user (standard Salesforce session). Guest users cannot invoke these — guest profiles do not have the DocGen_Admin / DocGen_User permset and the Schema-CRUD gate at every entry point throws DocGenException('Insufficient access to DocGen templates...') for unauthorized callers.
Authorization	Each invocable opens with an explicit Schema.sObjectType.<Object>.isAccessible/ isCreateable/isUpdateable() check before any SOQL or DML, per the v2.0 hybrid pattern documented in DocGen_Security_Architecture.md. Standard objects

Criterion	DocGen invocables
Input validation	(Account, Contact, etc.) are queried WITH USER_MODE so the user's CRUD/FLS is platform-enforced. Required fields are validated for null/blank/format before any database access. The Output Format Override is validated against an allowlist (PDF, Word, PowerPoint). The JSON Data input is parsed with <code>JSON.deserializeUntyped</code> inside a try/catch and surfaced via Error Message on failure (never thrown to the caller). Where applicable, Salesforce Id inputs are validated with <code>Id.valueOf()</code>.
PII handling	Document content is generated from records the calling user can already read (USER_MODE on user data). The <code>DocGenSignatureFlowAction</code> creates signer records that store an Email and Name from the Flow input — both are required to deliver a signing link to the named recipient, so they are not “extra” PII. Signer audit records are immutable and field-history tracked.
Trust audit	Every <code>@InvocableMethod</code> call that performs DML is captured by the standard Salesforce audit trail. The signature pipeline additionally writes a <code>DocGen_Signature_Audit__c</code> record per signer with timestamp, IP, user agent, consent flag, PIN verification timestamp, and final document SHA-256 hash.
Default-deny	All four invocables return a typed result object with Success and Error Message — they never throw uncaught exceptions so a Flow Decision element can branch on failure without putting the surrounding Flow into a fault state. This is documented in each invocable's <code>description=</code> attribute so the Flow Builder author understands the contract.
Idempotency	<code>DocGenFlowAction.generate</code> and <code>DocGenGiantQueryFlowAction.generateDocument</code> are idempotent for the same (template, record) pair — repeated invocations produce duplicate ContentVersions, which is the desired Flow-author-controlled behavior (they can deduplicate downstream). <code>DocGenSignatureFlowAction.generate</code> creates a fresh signature request each time and is intentionally non-idempotent (each invocation is a new legal signing event). <code>DocGenBulkFlowAction.generateBulk</code> enqueues a job; calling it twice enqueues two jobs (also intended).
Why none of these need to be agent actions	An admin in the subscriber org may choose to expose any of these as an agent action by creating a <code>GenAiFunction</code> that wraps the Flow. The package does not ship such a wrapper; the decision and the trust posture of “should an agent be allowed to autonomously generate a document or initiate a legal signing event for the running user” is intentionally left to the subscriber admin. If we ship a packaged agent action in a future release, it will be submitted for re-review with a <code>GenAiFunction</code> metadata file and an updated section here.

For the per-method security gates and SOQL/DML execution-mode rationale, see the “CRUD/FLS Enforcement Model” section of DocGen_Architecture_and_Usage.md and the per-method dispositions in SECURITY_REVIEW_RESPONSE_v2.md.

Question 6 — Unmanaged code and metadata dependencies

“If the solution depends on unmanaged code and metadata, share details about the dependencies. Unmanaged code and metadata include unmanaged flows, Apex, experience sites, prompt templates, agent topics, agent actions, and so on.”

Answer: N/A — no unmanaged dependencies.

DocGen v2.1.0 ships everything it needs inside the managed package:

- 42 Apex classes + 29 test classes (all namespaced as portwoodglobal__)
- 18 Lightning Web Components (all namespaced)
- 4 Visualforce pages (all namespaced)
- 11 custom objects (9 sObjects + 2 platform events, all namespaced)
- 4 permission sets (all namespaced)
- 2 sample Flows (DocGen_Generate_Account_Summary, DocGen_Welcome_Pack_New_Contact) shipped as admin-editable starting points
- 0 unmanaged Apex required in the subscriber org
- 0 unmanaged prompt templates required
- 0 unmanaged agent topics, agent actions, or GenAiPlugins required
- 0 unmanaged experience sites required (customer optionally creates a Salesforce Site for the guest-user signing page, but the page itself is in the package)

The customer’s optional post-install configuration (Org-Wide Email Address selection, brand-color/logo, Salesforce Site for signing) is metadata the **subscriber creates in their own org**, not unmanaged dependencies of the package.

Question 7 — Messaging for In-App and Web (MIAW) integration

“If the solution uses Messaging for In-App and Web (MIAW) integration, share details including the end-to-end use cases.”

Answer: N/A — DocGen does not integrate with MIAW.

DocGen has no Messaging for In-App and Web surface. No MIAW conversation handlers, no embedded service deployments, no in-app messaging channels are bundled with the package or required by it.

Question 8 — Third-party LLM API usage

“Does the solution use any third-party large language model (LLM) APIs directly? For example, it makes direct API calls to OpenAI APIs. If yes, revise the solution to use a trusted alternative before you submit it for review. Until further notice, such API calls aren’t allowed for AppExchange solutions. For information about trusted alternatives that Salesforce offers, see Models API Developer Guide and Learn the Basics of the Models API.”

Answer: NO — DocGen does not use any third-party LLM API.

DocGen v2.1.0 contains:

- 0 callouts to third-party APIs (OpenAI, Anthropic, Google AI, AWS Bedrock, etc.)
- 0 Remote Site Settings
- 0 Named Credentials
- 0 `Http.send()` invocations
- 0 calls to the Salesforce Models API
- 0 prompt templates, 0 `EinsteinGenerationsRequest` calls, no LLM-based features of any kind

The package is **100% native Apex** — no external services, no AI/ML APIs, no agent-or-LLM-driven features. All document generation logic is deterministic merge-tag substitution against Salesforce record data; all chart rendering is pure-Apex PNG rasterization (no external chart service); all signature flows are deterministic token-and-hash validation (no LLM).

Verified by:

- Salesforce Code Analyzer (Security + AppExchange rule selectors) — 0 callout sinks reported.
- Source-tree grep — `grep -rli "Http\..send\|HttpRequest\|HttpClient\|callout=true\|openai\|anthropic\|gemini\|bedrock\|huggingface" force-app/main/default/` returns no matches.

Conclusion

This questionnaire is non-applicable to Portwood DocGen v2.1.0. The package contains zero Agentforce elements. No further information is required from this document for the reviewer’s security review of the v2.1.0 submission.

For the actual security review of v2.1.0, the relevant documents are:

- `Listing_Describe_Solution.md / .pdf` (paste-ready field response)
- `Listing_Platform_Technology.md / .pdf` (paste-ready field response)
- `DocGen_Security_Architecture.md / .pdf` (long-form security architecture)
- `DocGen_Architecture_and_Usage.md / .pdf` (long-form feature inventory)
- `DocGen_Platform_Technology.md / .pdf` (long-form platform tech inventory)
- `DocGen_Code_Analyzer_Report.md / .pdf` + `CodeAnalyzer_Report.{csv,html,json}` (analyzer disposition + raw output)

- DocGen_False_Positive_Report.md / .pdf (Checkmarx CxSAST FP disposition with v2.0 hybrid pattern rationalizations)
- SECURITY_REVIEW_RESPONSE_v2.md / .pdf (per-finding map of v1.56 findings → v2.0 commits)
- README.md / .pdf (folder map + submission checklist)

All are in docs/appexchange/v2.1.0/ in the source repository at <https://github.com/Portwood-Global-Solutions/DocGen>.

Portwood Global Solutions — <https://portwood.dev>